Iron Stats: Continuous Biomechanical Tension Modeling, Physiological Fatigue Dynamics, and Multi-Objective Mesocycle Optimization

Lorenzo Zambardi
Department of Computer Science & Computational Exercise Modeling
https://github.com/Lorenzo1901/Iron-Stats

Technical Whitepaper — November 2026

Abstract

Conventional resistance-training analytics quantify training stress via coarse discrete scalars—raw set counts, repetition totals, and mechanical tonnage ($L \times R$). Such aggregates implicitly assume uniform mechanical tension across the entire Range of Motion (ROM), identical motor-unit recruitment regardless of proximity to muscular failure, and linear fatigue accumulation across repetitions. **Iron Stats** replaces these scalar heuristics with a unified computational model spanning three layers: (1) *continuous biomechanical tension fields*, parameterizing per-muscle resistance profiles over normalized ROM $t \in [0, 1]$ via area-normalized cubic Bézier curves; (2) *physiological load and fatigue dynamics*, modeling High-Threshold Motor Unit (HTMU) recruitment via an RPE-gated effective-repetition threshold, piecewise RIR-dependent concentric Time Under Tension (TUT) slowdown, and exponential central nervous system (CNS) fatigue; and (3) *constrained combinatorial optimization*, synthesizing balanced multi-day mesocycles via Monte Carlo scale calibration and scalarized Simulated Annealing. We present a numerical analysis comparing Lagrange interpolation, C^2 natural cubic splines, and Bernstein-basis cubic Bézier curves on empirical exercise profiles, proving why Bézier convex-hull containment and $\mathcal{O}(1)$ De Casteljau evaluation eliminate Runge divergence and non-physical negative tension while supporting exact $\mathcal{O}(h^2)$ trapezoidal magnitude conservation.

1 Introduction

Quantitative hypertrophy programming traditionally reduces a resistance-training session to discrete scalar summaries:

$$V_{\text{naive}} = \sum_s R_s, \qquad T_{\text{naive}} = \sum_s L_s R_s, \tag{1}$$

where R_s and L_s denote repetitions and external load in set s. Despite widespread adoption, these scalars exhibit three structural deficiencies:

1. **Biomechanical Blindness:** Moment arms and external resistance torque vary continuously across joint angles. Dumbbell Lateral Raises (`LR`), for example, peak in resistance near full abduction ($t \to 0.9$), whereas stretch-biased movements peak near $t \to 0$. Scalar tonnage treats both identically.
2. **Effort Invariance:** Two sets of 10 repetitions—one stopped at Rate of Perceived Exertion $\text{RPE} = 5$ (5 Reps in Reserve, $\text{RIR} = 5$) and one pushed to concentric failure ($\text{RPE} = 10$)—yield identical V_{naive}, despite markedly different HTMU recruitment and velocity loss.
3. **Linear Fatigue Assumption:** Systemic CNS and local tissue fatigue grow nonlinearly as repetitions approach failure, accompanied by progressive concentric velocity decay.

To address these gaps, **Iron Stats** models every exercise over a normalized continuous ROM domain $t \in [0, 1]$, integrates per-repetition physiological slowdown and exponential fatigue, and formulates mesocycle synthesis as a constrained multi-objective combinatorial optimization problem. All equations reported herein match the reference implementation in `python_research/analyzer/` (`constants.py`, `models.py`, `metrics.py`, `solver.py`).

2 Biomechanical Exercise Representation

In `models.py`, each exercise e is parameterized by a biomechanical tuple

$$\theta_e = \big(c_{\text{load}},\, f_{\text{base}},\, \mu_{\text{load}},\, q_{\text{load}},\, \text{is_isolation}\big), \tag{2}$$

where $c_{\text{load}} \in (0, 1]$ denotes articular load efficiency, $f_{\text{base}} \in [1, 10]$ is the intrinsic systemic fatigue coefficient, $(\mu_{\text{load}}, q_{\text{load}}) \in \mathbb{R}^+ \times \mathbb{R}$ maps nominal logged load L_{raw} to effective biomechanical load via

$$L = L_{\text{raw}} \cdot \mu_{\text{load}} + q_{\text{load}}, \tag{3}$$

and $\text{is_isolation} \in \{0, 1\}$ flags single-joint movements. (For instance, `LR` specifies $\mu_{\text{load}} = 2.0$ to convert per-dumbbell weight into bilateral load.)

Each exercise targets an anatomical sub-muscle set $\mathcal{M}_e = \{m_1, \ldots, m_k\}$ with contribution magnitudes $\mathbf{M}_e = [M_{e,m_1}, \ldots, M_{e,m_k}]^\top$. To conserve total mechanical work attribution across muscle compartments, `Exercise.__post_init__` enforces the partition-of-unity constraint

$$\sum_{m \in \mathcal{M}_e} M_{e,m} = 1.0, \tag{4}$$

automatically re-normalizing $M_{e,m} \leftarrow \text{round}(M_{e,m} / \sum_j M_{e,j}, 4)$ whenever $|\sum_m M_{e,m} - 1.0| > 10^{-4}$ (accounting for IEEE-754 floating-point tolerances in the 61-exercise JSON catalog).

3 Numerical Analysis of Tension Profiles

Representing muscle tension $y(x)$ across normalized ROM $x \in [0, 1]$ requires a functional basis that is parsimonious for interactive editing, smooth, numerically stable, and physically non-negative ($y(x) \geq 0$, as skeletal muscle cannot generate repulsive active tension). We evaluate three candidate numerical frameworks on the `LR` (*Lateral Deltoid*) reference profile defined in `defaultExercises.json` ($P_0 = (0.20, 0.00)$, $P_1 = (0.33, 0.30)$, $P_2 = (0.66, 0.80)$, $P_3 = (0.90, 1.00)$, $M = 0.90$).

3.1 Lagrange Interpolation and Runge Divergence

Given n sampled ROM-tension nodes $\{(t_i, y_i)\}_{i=1}^n$, the unique degree-$(n-1)$ global interpolating polynomial is

$$p_{n-1}(t) = \sum_{i=1}^{n} y_i \ell_i(t), \qquad \ell_i(t) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{t - t_j}{t_i - t_j}. \tag{5}$$

Although barycentric/Lagrange evaluation avoids directly inverting the ill-conditioned Vandermonde system $V\mathbf{a} = \mathbf{y}$ (which reaches $\kappa(V) = 2.01 \times 10^6$ already at $n = 9$), uniform nodes yield a Lebesgue constant $\Lambda_n = \max_{t \in [0,1]} \sum_{i=1}^n |\ell_i(t)| \sim \mathcal{O}(2^n)$. Consequently, the interpolation error bound $\|f - p_{n-1}\|_\infty \leq (1 + \Lambda_n) E_{n-1}(f)$ exhibits severe boundary oscillation (*Runge phenomenon*) in the resting stretch ($t < 0.2$) and contraction termination ($t > 0.85$) zones, as shown in Fig. 1.

Replacing equispaced nodes with Chebyshev–Gauss–Lobatto (CGL) nodes mapped to $[0, 1]$,

$$t_k = \frac{1}{2}\left(1 - \cos\frac{k\pi}{N-1}\right), \qquad k = 0, \ldots, N-1, \tag{6}$$

reduces Lebesgue growth to $\Lambda_N \sim \frac{2}{\pi} \ln N + \mathcal{O}(1)$ and suppresses edge blow-up. However, as Fig. 2 illustrates,

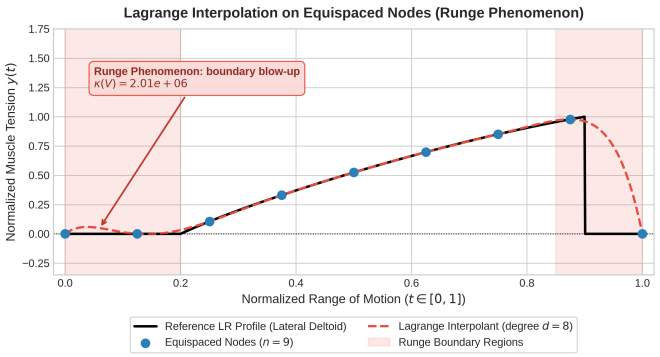


Figure 1: Lagrange polynomial interpolation ($n = 9$ equispaced nodes) on the `LR` Lateral Deltoid profile. Vandermonde conditioning reaches $\kappa(V) = 2.01 \times 10^6$, inducing boundary Runge oscillations.

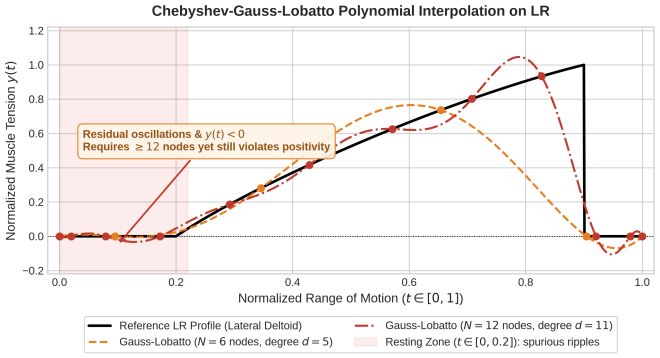


Figure 2: Chebyshev–Gauss–Lobatto interpolation ($N = 6, 12$) on `LR`. Boundary divergence is damped, but global coupling produces negative ripples in the resting zone $t \in [0, 0.2]$.

the flat resting region $t \in [0, 0.2]$ forces global polynomials to develop spurious ripples and non-physical negative dips ($y(t) < 0$) even at $N = 12$ nodes ($d = 11$), making user parameterization impractical.

3.2 C^2 Natural Cubic Splines

Piecewise natural cubic splines partition $[0, 1]$ into n subintervals $[t_k, t_{k+1}]$, enforcing C^2 interior continuity and zero boundary curvature $S''(t_0) = S''(t_n) = 0$. Solving the resulting tridiagonal linear system $A\mathbf{m} = \mathbf{d}$ via the Thomas algorithm requires only $\mathcal{O}(n)$ operations and minimizes the bending energy $\int_0^1 [S''(t)]^2 \, dt$.

Nevertheless, global C^2 coupling introduces non-physical undershoot ($y(t) < 0$) whenever a flat zero-tension plateau transitions into an ascending ramp (Fig. 3). Suppressing undershoot requires either a dense knot grid ($N \geq 10$) or relaxing continuity to C^1 monotone piecewise cubic Hermite interpolation (PCHIP), trading away parameter parsimony or second-derivative smoothness (Fig. 4).

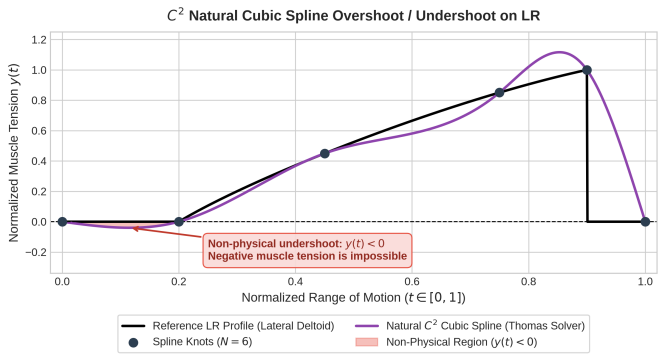


Figure 3: Natural C^2 cubic spline ($N = 6$ knots) on `LR`, exhibiting non-physical negative tension ($y(t) < 0$) across $t \in (0, 0.2)$ due to C^2 stiffness.

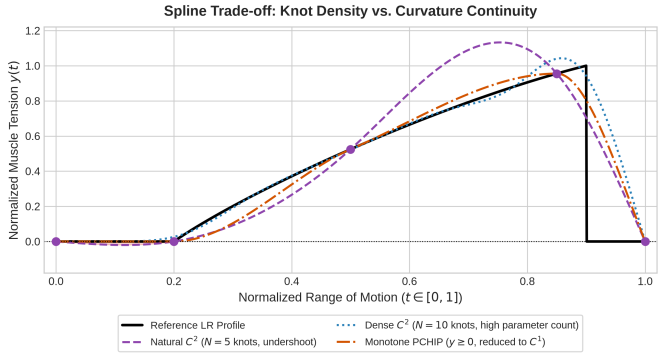


Figure 4: Spline trade-off on `LR`: sparse C^2 ($N = 5$) undershoots, dense C^2 ($N = 10$) inflates user parameters, and PCHIP sacrifices C^2 curvature.

3.3 Cubic Bézier Curves and Bernstein Guarantees

In `BezierProfile` (`models.py`), each muscle profile is represented parametrically by four 2D control points $\mathbf{P}_i = (x_i, y_i)^\top \in [0, 1]^2$, $i \in \{0, 1, 2, 3\}$, in the cubic Bernstein basis:

$$\mathbf{P}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix} = \sum_{i=0}^{3} \mathbf{P}_i B_{i,3}(t), \qquad t \in [0, 1], \tag{7}$$

where $B_{i,3}(t) = \binom{3}{i} t^i (1-t)^{3-i}$. Explicitly, in `models.py` ($x_{\text{sp}} = x_3 - x_0$, or 1.0 if $x_3 = x_0$):

$$x(t) = x_0 + x_{\text{sp}}\big(3(1-t)^2 t x_1 + 3(1-t)t^2 x_2 + t^3\big), \tag{8}$$

$$y(t) = (1-t)^3 y_0 + 3(1-t)^2 t y_1 + 3(1-t)t^2 y_2 + t^3 y_3. \tag{9}$$

Theorem 1 (Convex Hull Non-Negativity). *For all $t \in [0, 1]$, the cubic Bernstein polynomials satisfy $B_{i,3}(t) \geq 0$ and form a partition of unity $\sum_{i=0}^{3} B_{i,3}(t) = \big((1-t) + t\big)^3 \equiv 1$. Consequently, $\mathbf{P}(t) \in \text{Conv}(\{\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3\})$, and if $y_i \geq 0$ for all $i \in \{0, 1, 2, 3\}$, then $y(t) \geq 0$ for every $t \in [0, 1]$.*

Proof. Since $t \geq 0$ and $1 - t \geq 0$ on $[0, 1]$, each $B_{i,3}(t) \geq 0$. By the binomial theorem, $\sum_{i=0}^{3} \binom{3}{i} t^i (1-t)^{3-i} = 1$, so $y(t) = \sum_{i=0}^{3} B_{i,3}(t) y_i$ is a convex combination of $\{y_i\}$ and satisfies $\min_i y_i \leq y(t) \leq \max_i y_i$. Setting $y_i \geq 0$ guarantees $y(t) \geq 0$ identically. □

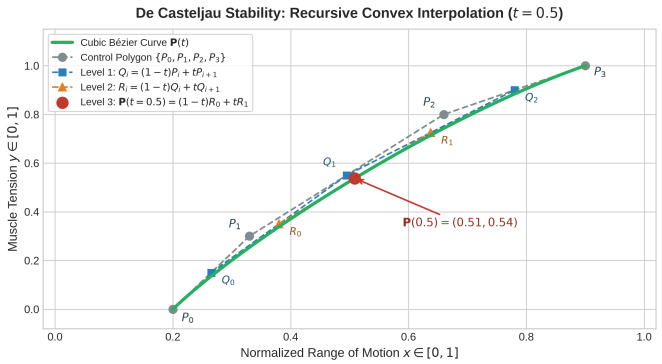


Figure 5: De Casteljau geometric construction at $t = 0.5$ on `LR`, requiring six division-free convex linear interpolations in $\mathcal{O}(1)$ time.

Table 1: Numerical properties of ROM tension approximations.

Method	**Params**	**Cost**	$y(t) \geq 0$	**Runge**
Lagrange (Equispaced)	n nodes	$\mathcal{O}(n^2)$	No	Severe
Lagrange (CGL)	$N \geq 12$	$\mathcal{O}(n \log n)$	No	Damped
Natural C^2 Spline	n knots	$\mathcal{O}(n)$	No	None
Cubic Bézier	**4 pts**	$\boldsymbol{\mathcal{O}(1)}$	**Yes**	**None**

Furthermore, $\mathbf{P}(t)$ is evaluated stably in $\mathcal{O}(1)$ time without division via De Casteljau's recursive convex linear interpolations (Fig. 5):

$$\begin{aligned} Q_i(t) &= (1-t)\mathbf{P}_i + t\mathbf{P}_{i+1}, & i &\in \{0, 1, 2\}, \\ R_i(t) &= (1-t)Q_i(t) + tQ_{i+1}(t), & i &\in \{0, 1\}, \\ \mathbf{P}(t) &= (1-t)R_0(t) + tR_1(t), \end{aligned}$$

where the segment $R_0(t)R_1(t)$ simultaneously supplies the exact tangent direction at $\mathbf{P}(t)$, and endpoints obey $\mathbf{P}'(0) = 3(\mathbf{P}_1 - \mathbf{P}_0)$ and $\mathbf{P}'(1) = 3(\mathbf{P}_3 - \mathbf{P}_2)$. When querying tension at a target ROM angle $x_{\text{target}} \in [x_0, x_3]$ with monotonic control abscissae $x_0 < x_1 < x_2 < x_3$, bisection solves $x(t^*) = x_{\text{target}}$ to tolerance $|x(t^{(k)}) - x_{\text{target}}| < 10^{-3}$ in at most 15 iterations in $\mathcal{O}(1)$ time. Fig. 6 and Fig. 7 summarize the Bernstein basis and the synoptic comparison across all three methods (Table 1).

3.4 Composite Trapezoidal Quadrature

To conserve muscle contribution magnitude $M_{e,m}$ under arbitrary control-point shapes, `BezierProfile.get_curve` discretizes $t \in [0, 1]$ on $R = 50$ uniform samples ($h = 1/(R-1)$) and integrates $y(x)$ via the composite trapezoidal rule:

$$I_{\text{trap}} = \sum_{k=0}^{R-2} \frac{y(t_k) + y(t_{k+1})}{2} \big(x(t_{k+1}) - x(t_k)\big), \tag{10}$$

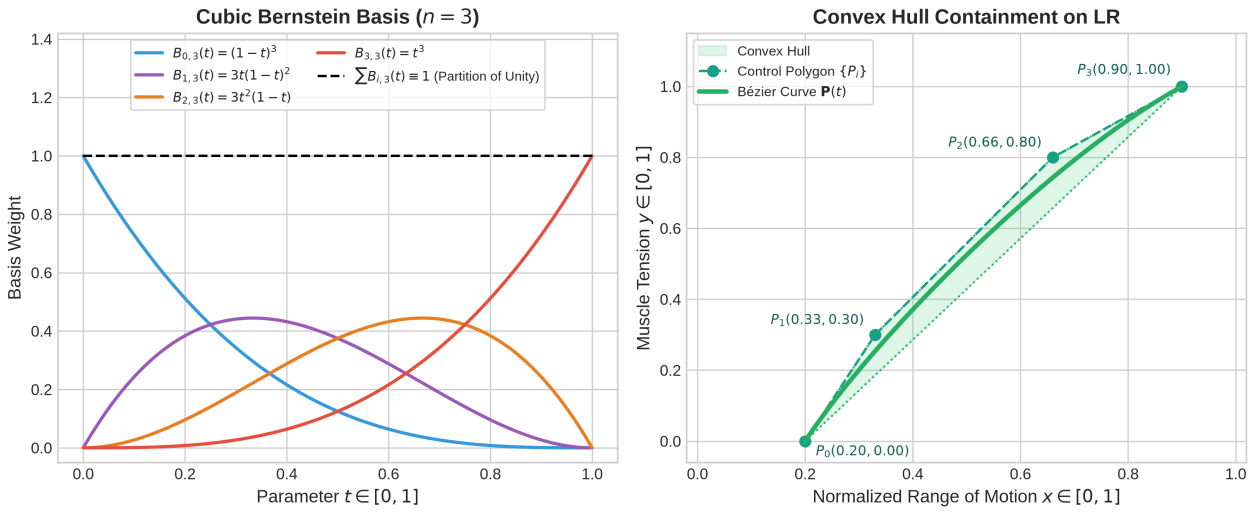


Figure 6: Left: Cubic Bernstein basis polynomials $B_{i,3}(t)$ verifying the partition of unity $\sum_{i=0}^{3} B_{i,3}(t) \equiv 1$. Right: Resulting `LR` curve enclosed strictly inside the convex hull of its four control points.

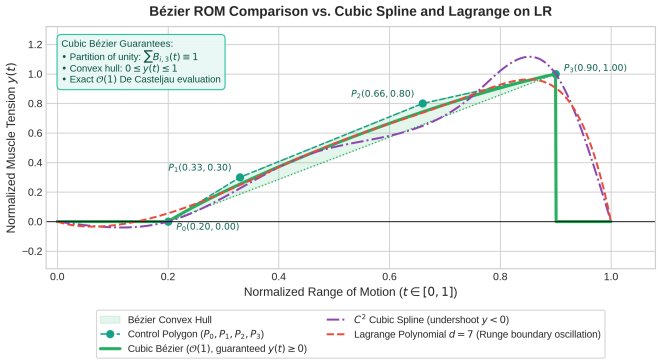


Figure 7: Synoptic comparison on `LR`: Cubic Bézier achieves $\mathcal{O}(1)$ evaluation and convex-hull positivity, whereas Lagrange ($d = 7$) suffers Runge oscillations and C^2 cubic splines violate $y(t) \ge 0$.

whose truncation error scales as $E_{\text{trap}} = -\frac{(x_3-x_0)^3}{12(R-1)^2} y''(\xi) = \mathcal{O}(h^2)$. The normalized curve is then rescaled as

$$B_{e,m}(t_k) = \begin{cases} y(t_k) \cdot \dfrac{M_{e,m}}{I_{\text{trap}}}, & I_{\text{trap}} > 0, \\ 0, & I_{\text{trap}} \le 0, \end{cases} \tag{11}$$

guaranteeing exact discrete area conservation $\sum_{k=0}^{R-2} \frac{B_{e,m}(t_k)+B_{e,m}(t_{k+1})}{2} \Delta x_k \equiv M_{e,m}$ and continuous convergence to $M_{e,m} + \mathcal{O}(h^2)$.

4 Physiological Dynamics & Optimization

4.1 Effective Volume, Dynamic TUT, and CNS Fatigue

In `constants.py` and `models.py` (`SetData`), each set contains n_b base repetitions, n_a assisted repetitions ($\alpha_{\text{ass}} = 0.5$), and n_p partial repetitions ($\alpha_{\text{part}} = 0.33$) at RPE $\in [5, 10]$ (default 9.0). With effective base reps $n_{b,\text{eff}} = \min(n_b, \max(0, \text{RPE} - 4.0))$, total and HTMU-effective repetitions are:

$$R_{\text{tot}} = n_b + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p, \tag{12}$$

$$R_{\text{eff}} = n_{b,\text{eff}} + \alpha_{\text{ass}} n_a + \alpha_{\text{part}} n_p. \tag{13}$$

Thus, base repetitions only recruit high-threshold motor units when RIR < 6 (RPE > 4.0).

In `metrics.py` (`calculate_set_tuts`), given execution tempo (c, p_s, e, p_l) (default 1-0-2-0 s) with nominal duration $\tau_0 = c + p_s + e + p_l$, each base repetition $j \in \{0, \ldots, n_b - 1\}$ experiences instantaneous $\text{RIR}_j = (10.0 - \text{RPE}) + (n_b - 1 - j)$ and concentric velocity slowdown $\Delta c(\text{RIR}_j)$:

$$\Delta c(\text{RIR}_j) = \begin{cases} 1.60\,\text{s}, & \text{RIR}_j < 1.0, \\ 1.00\,\text{s}, & 1.0 \le \text{RIR}_j < 2.0, \\ 0.60\,\text{s}, & 2.0 \le \text{RIR}_j < 3.0, \\ 0.35\,\text{s}, & 3.0 \le \text{RIR}_j < 4.0, \\ 0.15\,\text{s}, & 4.0 \le \text{RIR}_j < 5.0, \\ 0.00\,\text{s}, & \text{RIR}_j \ge 5.0, \end{cases} \tag{14}$$

yielding per-rep TUT $\tau_j = \tau_0 + \Delta c(\text{RIR}_j)$ for base reps, τ_0 for assisted reps, and $0.5\tau_0$ for partial reps. Effective TUT sums over the last $\lfloor \min(n_b, \max(0, \text{RPE} - 4)) + 0.5 \rfloor$ base reps plus all assisted and partial reps.

In `calculate_set_fatigue`, per-rep effort $\mathrm{RPE}_j = \max(0, 10.0 - \mathrm{RIR}_j)$ drives an exponential CNS fatigue multiplier $\mu(\mathrm{RPE}_j) = 1.1^{\mathrm{RPE}_j - 7.5}$ (or 1.0 when RPE $= 0$):

$$\Phi_{\text{set}} = F_{\text{ex}} K_{\text{load}} \Bigg[\sum_{j=0}^{n_b - 1} \tau_j \mu(\mathrm{RPE}_j) + \alpha_{\text{ass}} 1.1^{-0.5} n_a \tau_0 + \alpha_{\text{part}} n_p \tau_0 \Bigg], \tag{15}$$

where $F_{\text{ex}} = f_{\text{base}}$ and $K_{\text{load}} = c_{\text{load}}$. All set-level volume, tonnage (LR_{tot}), weighted tonnage ($LR_{\text{tot}} c_{\text{load}}$), TUT, and fatigue totals are projected onto queried macro- or sub-muscles via $M_{e,m}$ (`calculate_metrics`).

4.2 Constrained Multi-Objective Combinatorial Solver

In `solver.py` (`WorkoutSolver`), a candidate mesocycle $x \in \mathcal{X}$ assigns D training days, each containing $N_d \in [N_{\min}, N_{\max}]$ exercises, $S_{d,e} \in [S_{\min}, S_{\max}]$ sets, reps $r \in [r_{\min}, r_{\max}]$, RPE $\in \{7.0, 8.0, 8.5, 9.0, 10.0\}$, and stretched-ROM partials $p \in \{0, 2, 4, 6\}$ (enabled when RPE ≥ 9.0). Cumulative muscle tension across the mesocycle is computed as

$$V_m(t_k) = \sum_{d=1}^{D} \sum_{e \in x_d} B_{e,m}(t_k) \big(n_{b,e} + n_{p,e}\, \chi_{\text{part}}(t_k) \big), \tag{16}$$

where $\chi_{\text{part}}(t_k) = \mathbb{I}[t_k \leq 0.3333]$ captures stretch-mediated hypertrophy stimulus from partial repetitions restricted to the initial third of ROM. Letting $\hat{V}_m(t_k) = V_m(t_k) / \max_r V_m(t_r)$, seven dimensionless error metrics $\mathcal{I} = \{\text{curve}, \text{vol}, \text{ton}, \text{tut}, \text{distr}, \text{variety}, \text{balance}\}$ evaluate mesocycle quality:

$$E_{\text{curve}} = \frac{1}{|\mathcal{M}_{\text{tr}}| R} \sum_{m \in \mathcal{M}_{\text{tr}}} \sum_{k=0}^{R-1} \big(\hat{V}_m(t_k) - T_m(t_k) \big)^2, \tag{17}$$

$$E_q = \left(\frac{X_{q,\text{eff}}}{X_{q,\text{tot}}} - \rho_q^* \right)^2, \qquad q \in \{\text{vol}, \text{ton}, \text{tut}\}, \tag{18}$$

$$E_{\text{distr}} = \frac{1}{2} \sum_m \big(p_m^{\text{actual}} - p_m^{\text{target}} \big)^2, \tag{19}$$

$$E_{\text{variety}} = \left(1 - \frac{|\mathcal{U}(x)|}{N_{\text{tot}}(x)} \right)^2, \tag{20}$$

$$E_{\text{balance}} = \frac{1}{2} \left(\frac{\mathrm{Var}_d(F_d)}{(F_{\max}/2)^2} + \frac{\mathrm{Var}_d(V_d)}{(V_{\max}/2)^2} \right), \tag{21}$$

where $T_m \in \{\text{constant}, \text{sigmoid}, \text{inv_sigmoid}, \text{linear}\}$ and default $\rho_q^* = 0.70$. Prior to optimization, `calibrate()` samples $K = 500$ random states to compute scale factors $s_i = 2\,\mathbb{E}[E_i]$. `solve()` then minimizes the scalarized objective

$$\min_{x \in \mathcal{X}} \; J(x) = 10^4 \sum_{i \in \mathcal{I}} w_i \frac{E_i(x)}{s_i} + 10^3\, P_{\text{dup}}(x), \tag{22}$$

where $P_{\text{dup}}(x) = 150 \cdot N_{\text{dup}}(x)$ penalizes same-day duplicate exercises, using Simulated Annealing with eight discrete mutation operators, exponential cooling $T_{t+1} = \gamma T_t$ ($\gamma = (0.01/T_0)^{1/N_{\text{iter}}}$), and Metropolis acceptance $\exp(-\Delta J / T_t)$.

5 Conclusions & Software Architecture

Iron Stats demonstrates that replacing scalar training logs with Bernstein-basis Bézier tension fields, RIR-aware concentric slowdown, exponential fatigue modeling, and Monte Carlo calibrated Simulated Annealing resolves fundamental biomechanical and numerical pathologies present in polynomial and spline approximations. The architecture maintains strict mathematical parity between the reproducible Python research package (`python_research/analyzer/`, packaged via `pyproject.toml` and `requirements.txt` with `numpy` as sole runtime dependency) and the interactive React/Vite/Capacitor client (`frontend/`), which executes live logbook parsing, interactive Bézier control-point editing, and client-side mesocycle generation. Future work will focus on collecting longitudinal training datasets to empirically fit objective weights w_i and integrating inertial measurement unit (IMU) or computer-vision kinematics to refine exercise-specific $B_{e,m}(t)$ profiles.